

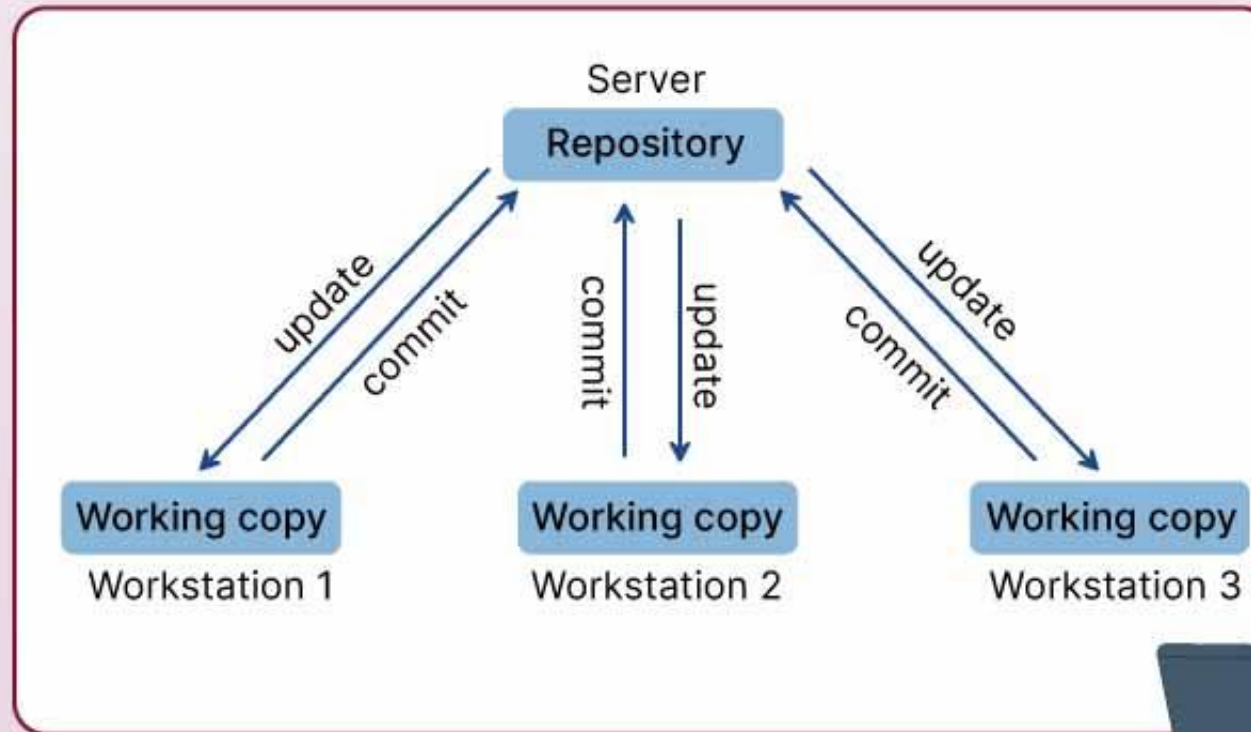
Module 2



Contents

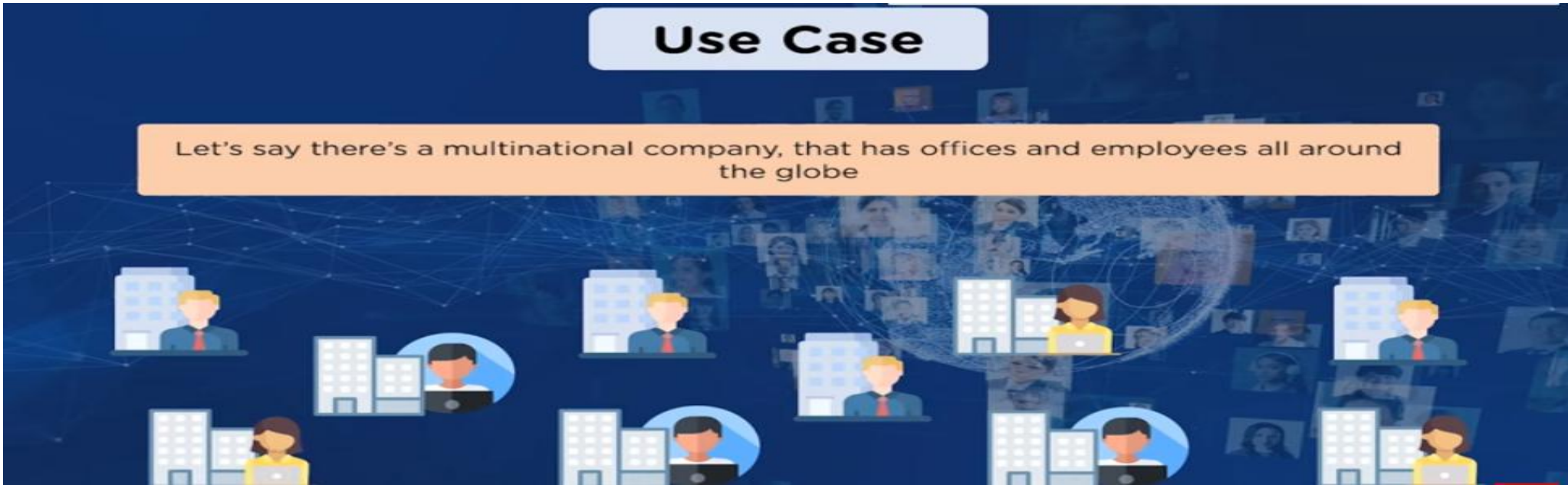
- **Version Control**
- **Continuous Integration,**
- **Continuous Testing,**
- **Configuration Management,**
- **Continuous Deployment,**
- **Containerization,**
- **Continuous Monitoring,**
- **Tool pipelining**

Version control in DevOps



Use Case

Let's say there's a multinational company, that has offices and employees all around the globe



Problems that may arise:

Collaboration



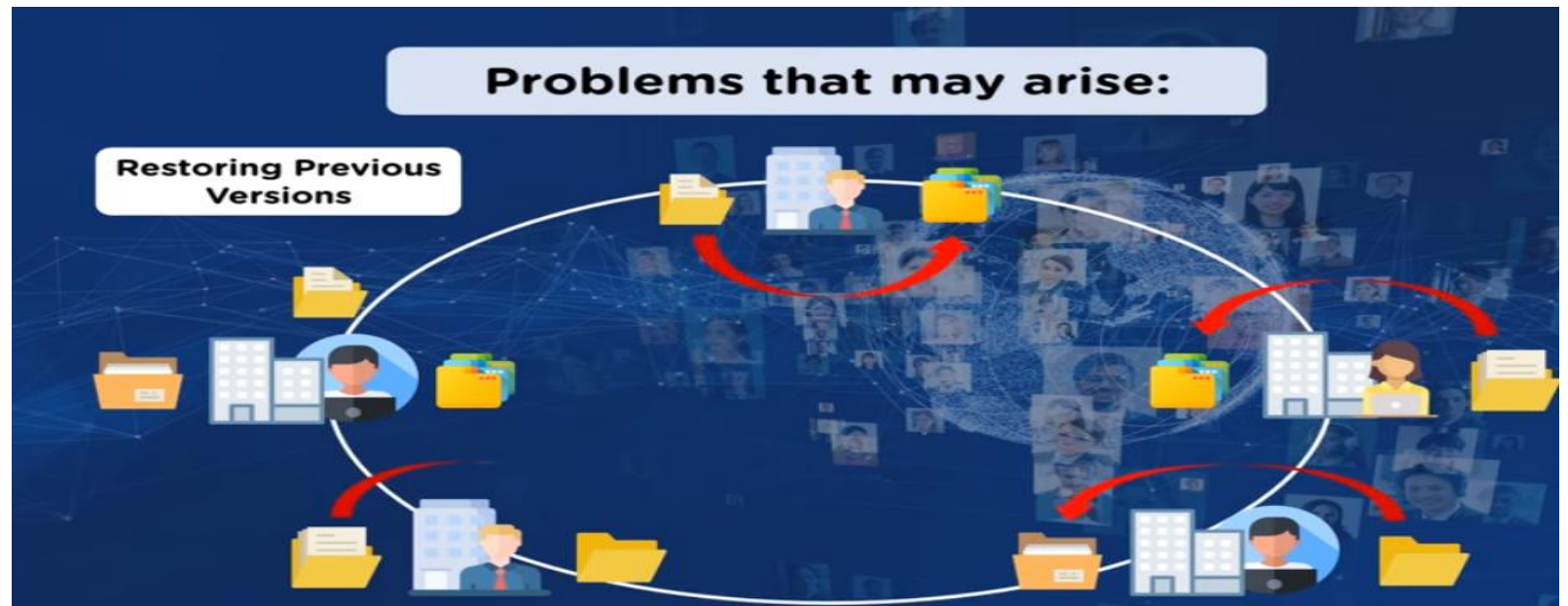
Problems that may arise:

Storing Versions



Problems that may arise:

Restoring Previous Versions



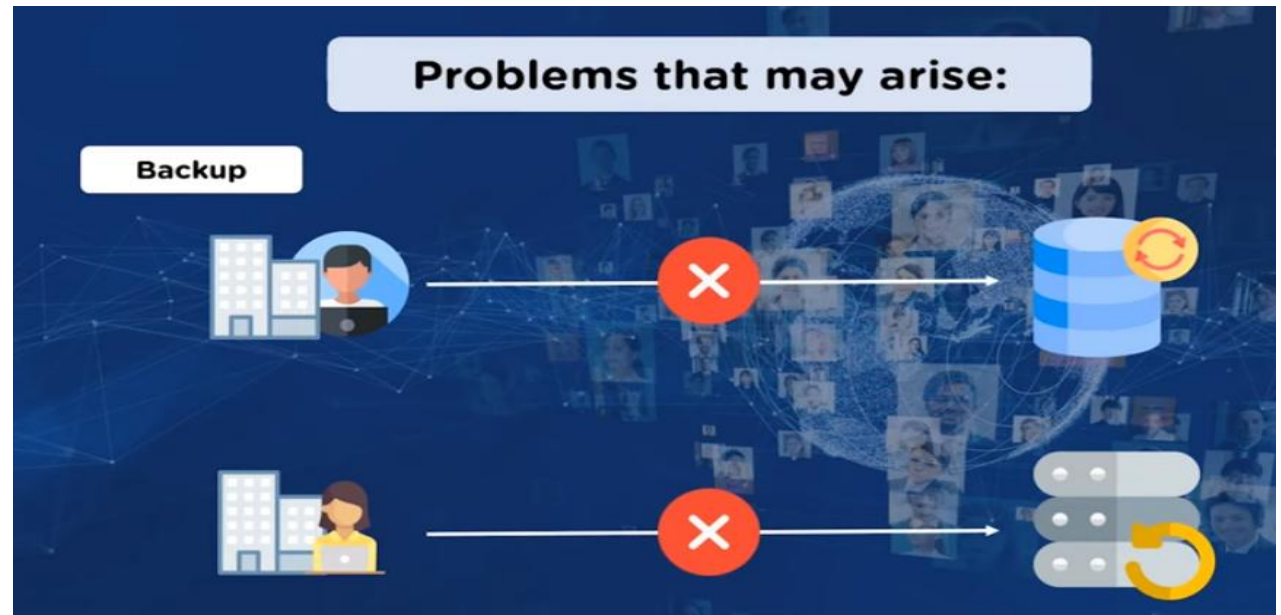
Problems that may arise:

Figuring out
what happened



Problems that may arise:

Backup



What could solve all these problems?

All these problems can be solved with the help of a

“Version Control System”

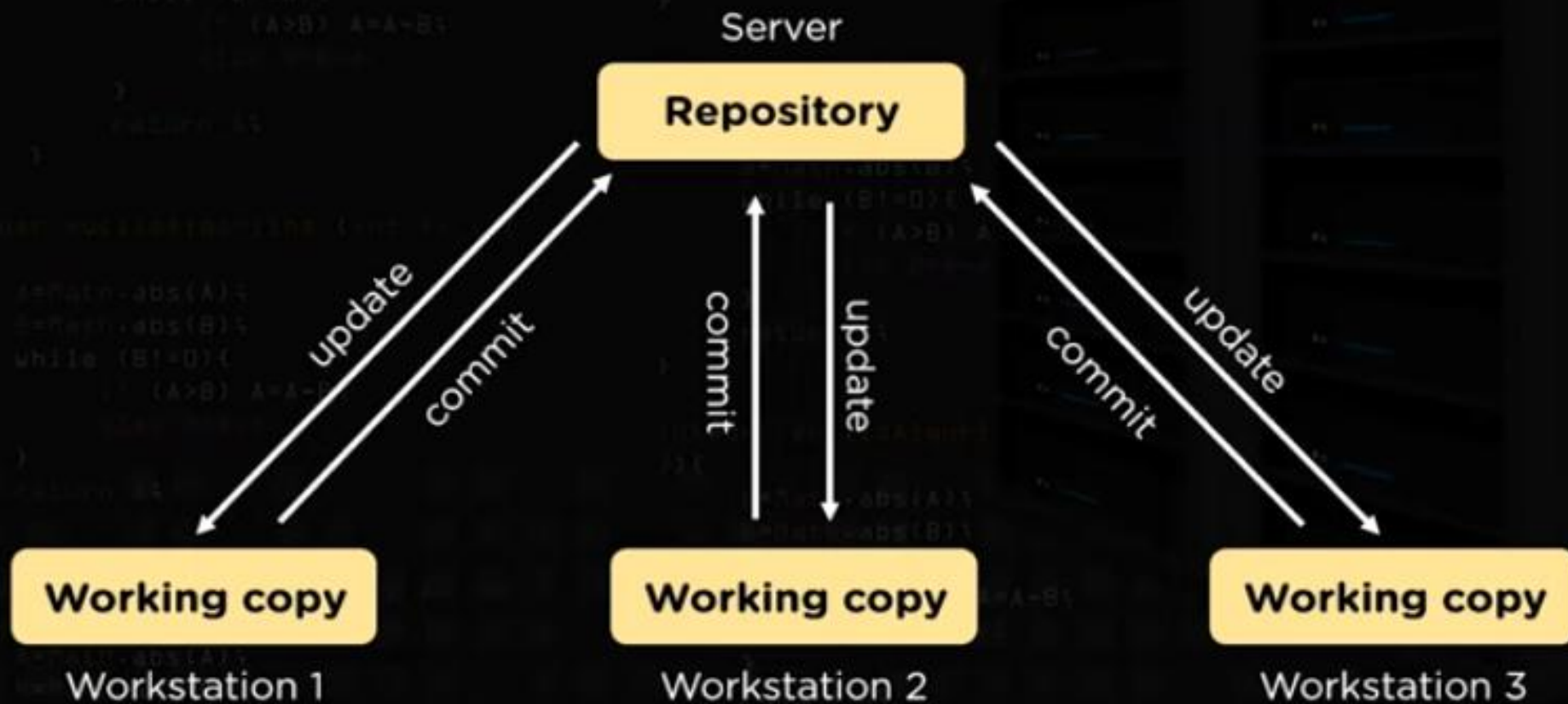


What is Version Control?

- A **Version Control** System records all the changes made to a file or set of files, so a specific version may be called later if needed
- The system makes sure that all the team members are working on the latest version of the file



What is Version Control?



Version Control

Version Control in DevOps

Version Control System (VCS) is a tool that helps track changes to source code or files over time

It enables multiple people to collaborate on a project, revert to earlier versions, and identify who made specific changes

Version Control System (VCS)

way to make changes to files without worrying about something that will get lost or things will fall out of the flow

helps to manage the changes done during development process in a project

offers backup and history of any changes for any files line-by-line

Version Control *contd..*

Google Docs

- **Multiple users can edit a document,**
- **track changes,**
- **see the revision history,**
- **restore older versions.**

Version control in software works similarly but with more power and flexibility.

How do we Version Control them?

- Earlier, version control used to happen manually in the absence of tools,
- where the team used to save a physical copy of back up and then make the changes.
- These days lot of sophisticated version controlling tools
- automatically takes care of version controlling by managing the check-ins and check-outs by multiple people simultaneously
- enable the team to view the difference between versions very easily
- Helps to debug the cause of failure very fast.

Centralized vs Distributed VCS

Centralized VCS (CVCS)

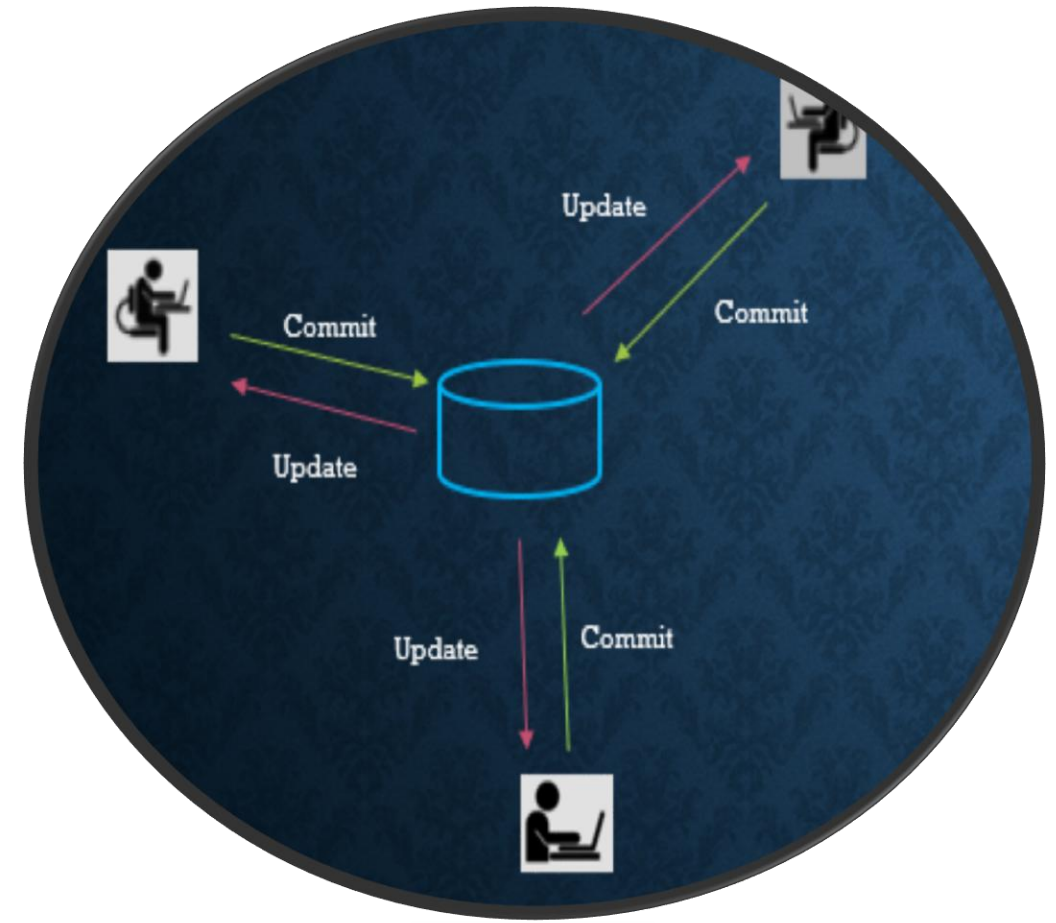
A single central server contains all the versioned files

Developers get the latest version commit directly to this server

Example: Office document stored on a shared network drive. If the server goes down, nobody can access it

Centralized
CVCS

Tool Examples:
Subversion (SVN),
Perforce



Centralized vs Distributed VCS contd..

Distributed VCS (DVCS)

Distributed VCS (DVCS)

Every developer has a local copy (clone) of the entire repository

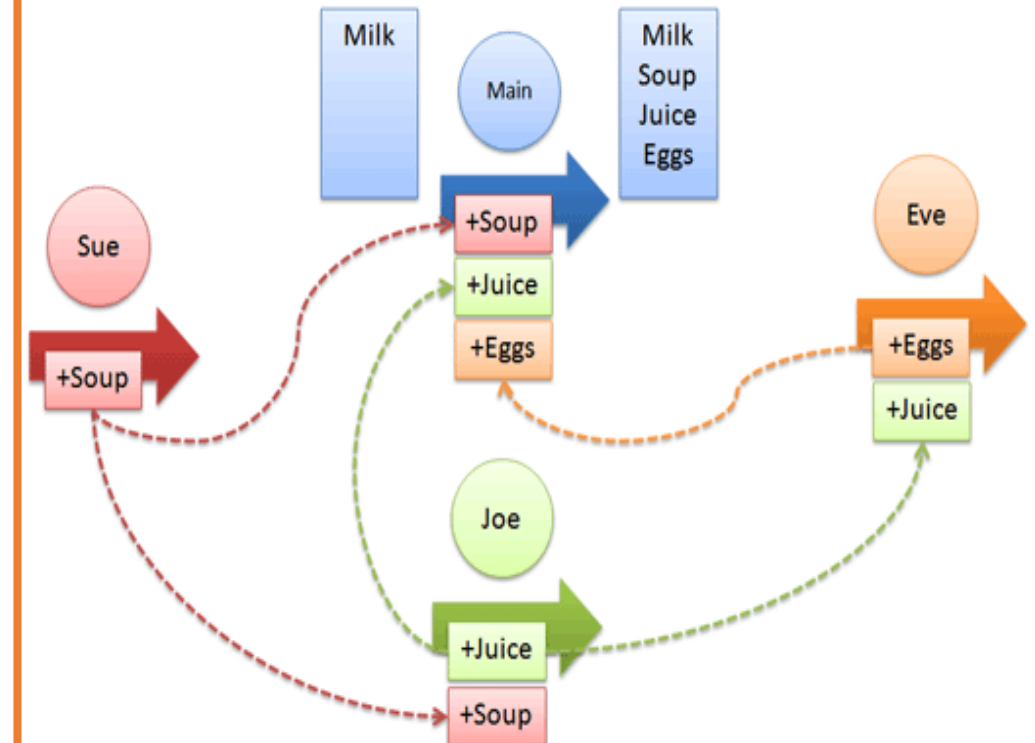
Changes can be committed locally and then pushed to a shared central repo

Distributed DVCS

Every team member has a copy of the document. They can work offline and sync changes when the server is available

Tool Examples:
Git,
Mercurial

Distributed VCS



Benefits of using Version Control (VCS)

- Provides a complete history of every file, made by the individuals over the years.
- protects the source code from any accidental changes, human mistakes, and unintended circumstances.
- The facility of Branching and Merging.
- helps to understand the difference between various versions and also the purpose of making the changes.
- beneficial in case of large distributed teams
- ensures that Dev and Ops are in there in the source control

Git as the Industry Standard

- **Git** is a free, open-source distributed VCS created by Linus Torvalds in 2005 (for Linux kernel development).
- It is fast, reliable, flexible, and widely adopted.
- Supports branching, merging, reverting, history tracking, and collaboration.
- Example:
- Git is used by Google, Microsoft, Netflix, and Facebook to manage their massive codebases.
- Microsoft migrated the entire Windows codebase (over 3.5 million files) to Git with the help of a tool called GVFS.

Some Popular Version Control Tools in DevOps

Feature	Git	GitHub
What it is	A distributed version control system (software/tool)	A web-based hosting platform for Git repositories
Where it's used	Installed locally on your computer	Used online (via browser or desktop client)
Internet needed?	No — you can use Git offline	Yes — GitHub is an online service
Primary purpose	Tracks changes in code/files, supports branching and merging	Helps developers share code, collaborate, and manage Git projects
Functionality	Manage versions, commit, branch, merge locally	Host repos, pull requests, issues, project management, CI/CD pipelines
Storage	Stores repos locally	Stores Git repositories in the cloud
Teamwork	Supports collaboration with manual setup (e.g., via email or Git server)	Built-in features for easy collaboration (pull requests, reviews, etc.)
Example Commands / Actions	git init, git add, git commit, git push	Create repo, fork repo, raise pull request, manage issues



Some Popular Version Control Tools in DevOps contd...

GitLab

- GitLab is an open-source Version Control System, written in Ruby and Golang.
- It comes with features like in integrated project, a project website, etc.
- One can automatically test and deliver the code using the Continuous Integration (CI) facility of GitLab.
- GitLab is repository management tool hosted on the free hosting service GitLab.com.
- It is easy to use to link projects via GitLab API.
- It works with various OS like Windows, Linux, OSX, etc.



Some Popular Version Control Tools in DevOps contd...

BitBucket

- BitBucket is a paid Version Control System.
- Its a part of the Atlassian's software suite.
- It offers features like code branches, in-line commenting and discussions and pull requests.
- BitBucket is specifically developed for the professional teams.
- It not just enables users to code but also to manage and collaborate on GIT projects.
- It can be deployed on the local server as well as on the cloud.



Some Popular Version Control Tools in DevOps contd...

Perforce

- Perforce is an open-source enterprise version control tool.
- Here, users connect to a shared file repository.
- Perforce applications are used to transfer files between the file repository and individual user workstations.
- It delivers version control through its **HelixCore**.
- **Handling very large codebases**
- **Managing binary assets (like videos, graphics, CAD files)**
- **Being used in large enterprises like game development, semiconductor design, and aerospace**



Some Popular Version Control Tools in DevOps contd...

Mercurial

- Mercurial (hg) is a Distributed Version Control System (DVCS) like Git but designed to be simpler, more user-friendly, and performance-oriented for large projects.
- It was popular before Git took over the market and is still used in some large projects (e.g., Mozilla Firefox used to use Mercurial).
- As compared to Git, developers find Mercurial easy to setup and use
- Its mostly developed in Python.



Continuous Integration in DevOps

Think-Pair-Share Activity

What do you think happens when multiple developers work on the same codebase without coordination?

- **Learners:** 2 minutes to think, then
- 2 minutes to discuss with a peer
- **outcomes:** Merge conflicts, bugs, instability, deployment delays, etc.

Definition and Importance

- Is a DevOps practice where developers frequently merge code changes into a shared repository, which then triggers automated builds and tests.
- This ensures early detection of integration issues.

Merge Conflicts

When multiple developers modify the same code/file, Git or other VCS may not be able to automatically resolve differences.

Example:

Two developers, A and B, are working on the same file

LoginService.java.

- A renames a method from `validateUser()` to `authenticateUser()`.
- B adds new logic to the old `validateUser()` method.

When both push their changes, the CI pipeline detects the **conflict**, prompting them to resolve it before the build continues.

Broken Builds

A newly pushed change may not compile or pass build scripts.

Example:

Developer D forgets to import a required module or uses a method that doesn't exist anymore.

The CI build step fails with **Compile Error: cannot find symbol**.

The team is notified immediately.

Dependency Conflicts

code formatting or naming conventions are not followed

Example:

Developer E commits Python code without following **PEP8** standards.

CI tool like **flake8** or **pylint** throws errors such as:

E302 expected 2 blank lines, found 1.

CI fails the build until the formatting is fixed.

Code Style Violations / Linting Errors

One developer may update or add a new library/package that breaks compatibility for others.

Example:

Developer C updates a library version in **pom.xml** from **Spring Boot 2.5.0** to **2.7.0**, which introduces breaking changes.

CI runs the build and throws errors due to **API deprecation or changed annotations**, ensuring the problem is caught before deployment.

Configuration Mismatches

Example:

Differences in development and test environment setups can cause unexpected errors.

An app works locally, but CI pipeline fails because **DATABASE_URL** environment variable is not set in the CI environment.

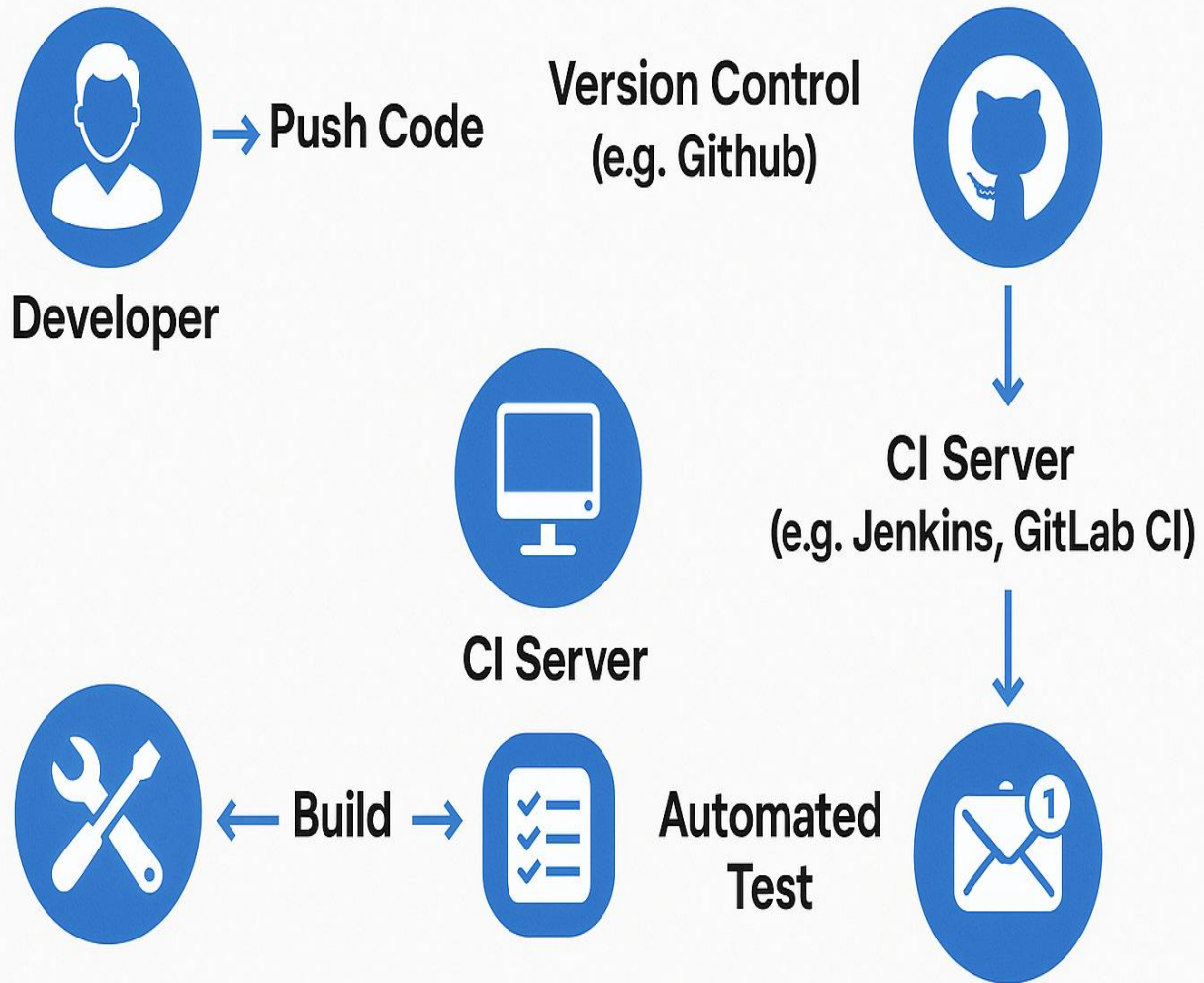
CI shows error:

Error: DATABASE_URL not found.

This prompts proper environment setup for consistent deployment.

How Does Continuous Integration Works?

Continuous Integration in DevOps



- **Developer** writes code and **pushes** it to a shared repository (like GitHub).
- The **Version Control System** stores the code.
- A **CI Server** (like Jenkins or GitLab CI) checks for new code.
- It **builds** the code automatically.
- Then it runs **automated tests** to check for errors.
- If everything works, it sends a **notification** (like an email) to the team.

CI Workflow

1. Developer Commits Code

- Each time a developer writes new code or modifies existing code, they commit it to the shared version control system (e.g., Git).
- **Example:** A frontend developer commits a new [login page](#) component to a GitHub repo.

2. Automated Build and Test Triggered

- A CI tool (like Jenkins or GitHub Actions) detects the commit.
- It automatically runs:
 - A **build** (compiling code)
 - **Automated tests** (unit, integration, etc.)
- **Example:** Jenkins triggers a build and runs unit tests for the login feature. If the login breaks existing functionality, the tests will fail.

CI Workflow Contd..

3. Feedback Sent

- The CI tool sends instant feedback to the developer.
- Feedback may include:
 - Build success/failure
 - Test results
 - Code quality issues
- **Example:** A Slack notification or email alerts the developer:
"Build #124 failed – 2-unit tests failed in login.js"

Benefits

Early Bug Detection

- Bugs are caught before they reach production.
- Developers know immediately when something breaks.
- **Example:** A bug in a new payment module is caught within minutes of commit because automated tests fail.

Faster Feedback

- Developers receive test/build results quickly (within minutes).
- Encourages regular, smaller commits and quick improvements.
- **Example:** A mobile app team commits code 10–15 times a day; CI allows them to identify and fix issues in real-time.

Better Code Quality

- Frequent builds and tests ensure stable, reliable code.
- Encourages coding discipline and documentation.
- **Example:** Static code analysis tools integrated in the CI pipeline enforce code style and quality standards automatically (e.g., ESLint, SonarQube).

CI Pipeline in Action

Real-Life Example (CI Pipeline in Action)

A team at **Zomato** develops a food delivery web app:

1. Developer commits **new cart** logic to GitHub.
2. GitHub Actions runs a pipeline:
 1. Builds the app
 2. Runs Cypress end-to-end tests
3. Tests fail due to a broken button click.
4. Developer gets a Slack notification and fixes the bug.
5. After re-commit, tests pass, and code is ready for deployment.

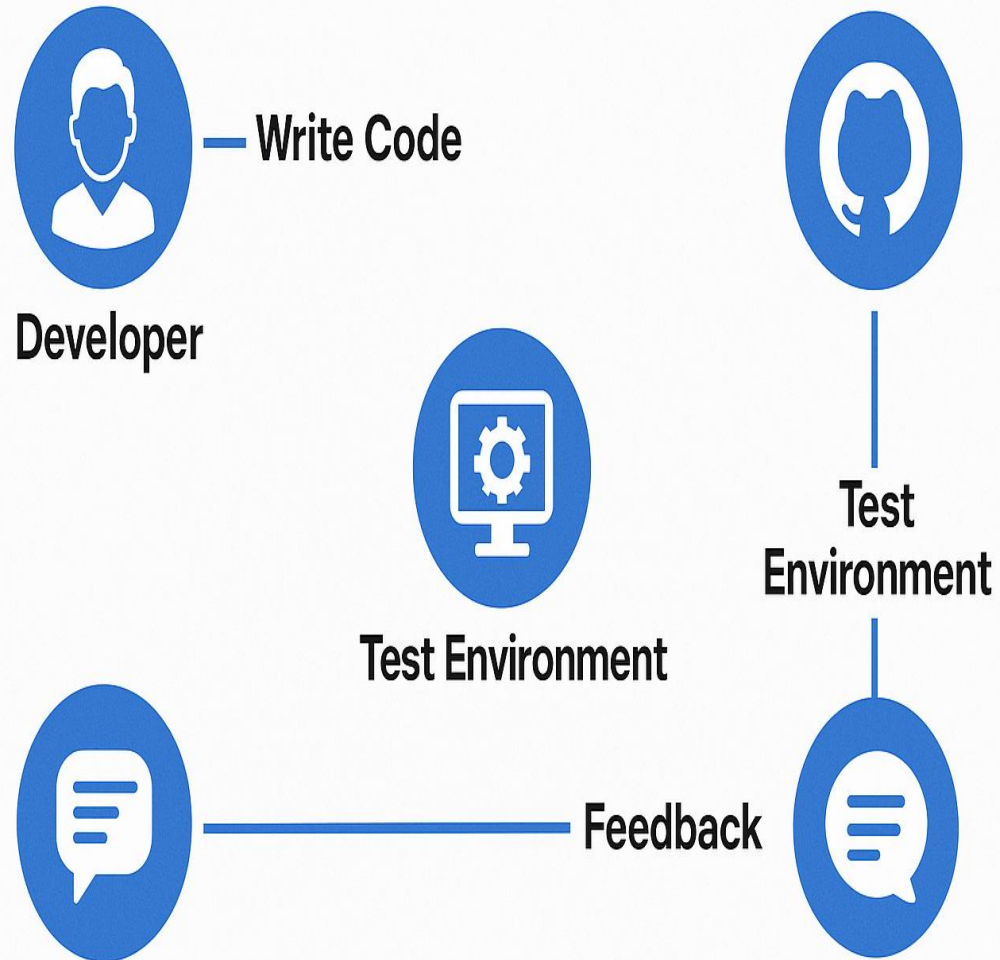
Continuous Testing in DevOps

Continuous Testing

- Execution of automated tests that are carried out at regular intervals every time code changes are made
- Involves testing the software at every stage of the software development life cycle.
- Goal is evaluating the quality of software at every step of the Continuous Delivery Process by testing early and testing often.
- Involves stakeholders like Developer, DevOps, QA and Operational system.

How Continuous Testing Works

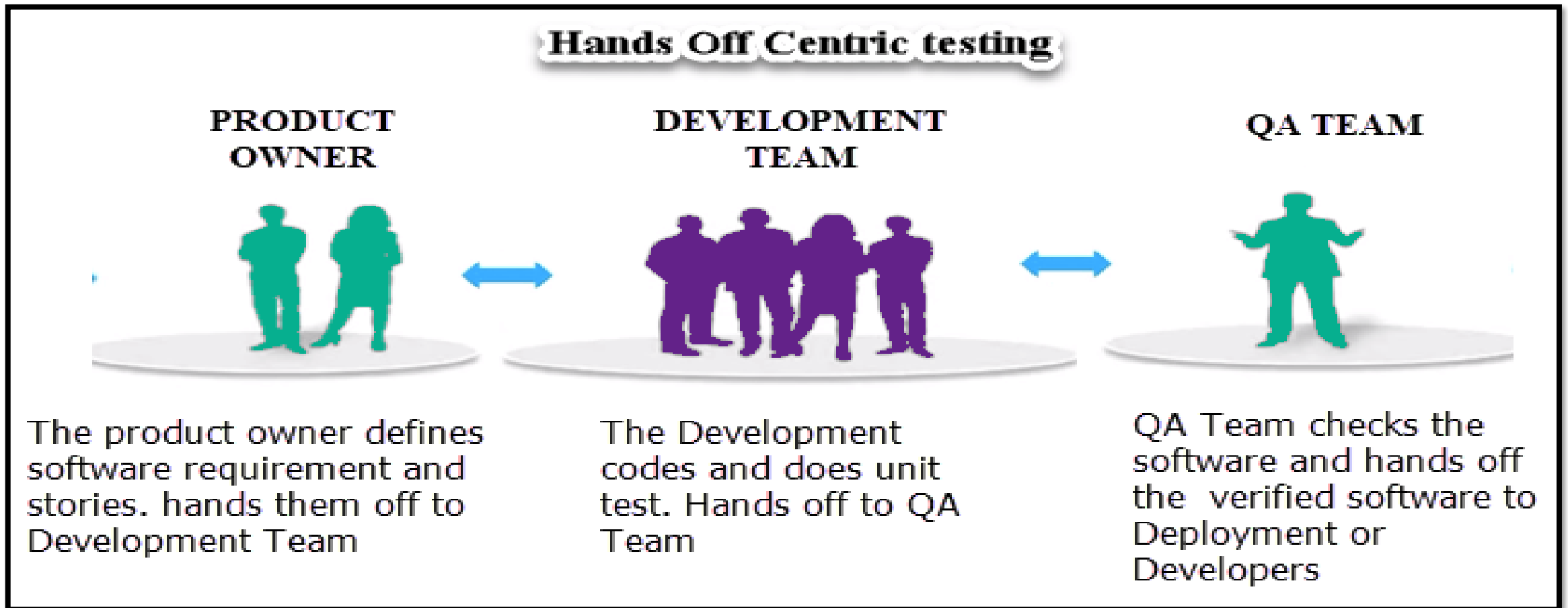
Continuous Testing in DevOps



- **Developer writes code.**
- Code is pushed to a shared system (like GitHub).
- Code is tested **automatically** in a **test environment**.
- Results are shared quickly as **feedback**.
- Developer uses the feedback to fix or improve the code.

How is Continuous Testing different?

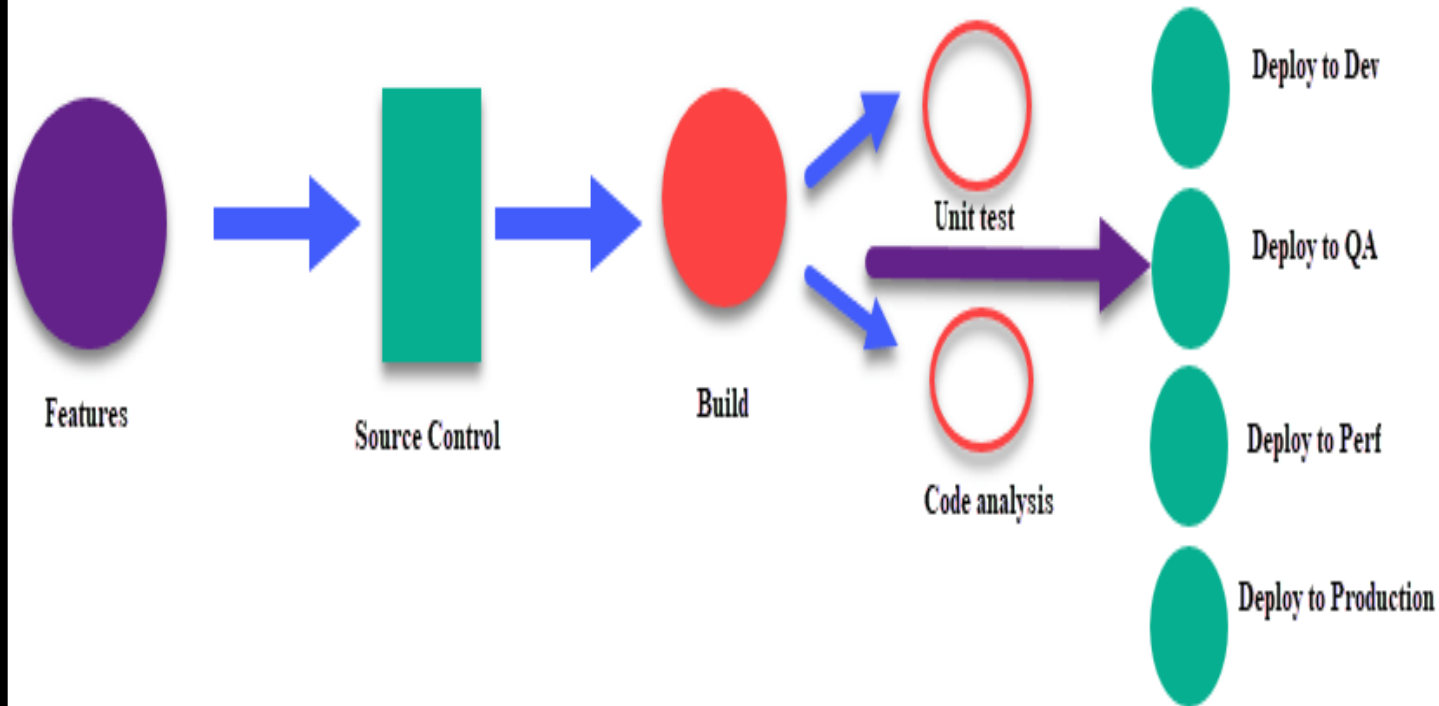
The old way of testing was hand off centric.



How is Continuous Testing different? Contd..

Continuous DevOps Process

Continuous Testing



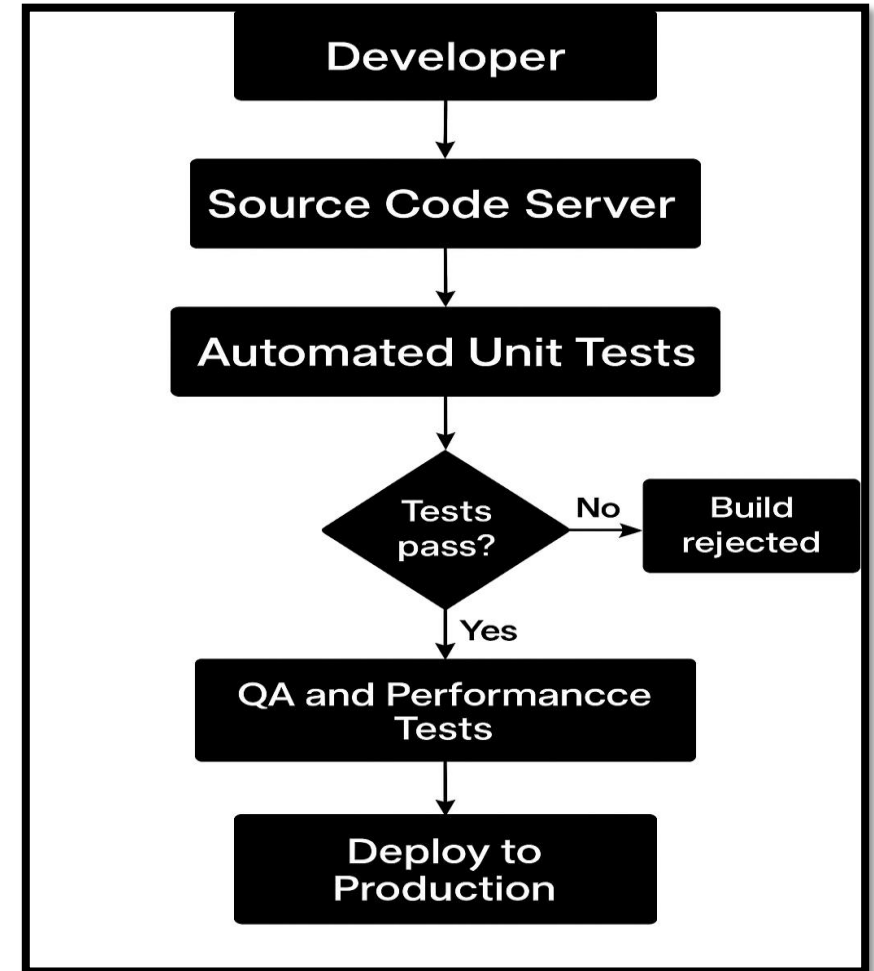
1. Features are developed by the team.
2. Code is saved in Source Control (like Git).
3. A Build is created from the code.
4. Automated Unit Tests and Code Analysis check the build.
5. If successful, the build is sent to different stages:

- Dev (Developer Testing)
- QA (Quality Assurance Testing)
- Perf (Performance Testing)
- Production (Live for users)

The code is continuously developed, delivered, tested and deployed.

Continuous Testing contd..

- For Example, whenever a developer checks the code in the Source Code Server like Jenkins automated set of unit tests are executed in the continuous process.
- If the tests fail, the build is rejected, and the developer is notified.
- If the build passes the test, it is deployed to performance, QA servers for exhaustive functional and load tests.
- The tests are run in parallel.
- If the tests pass, the software is deployed in production.



How to do Continuous Testing

- Using tools to generate test automation suite from user stories/requirements
- Create Test Environment.
- Copy and anonymize production data to create test data bed
- Use service virtualization to test API
- Parallel performance testing

Continuous Testing Tools

1) QuerySurge

is the smart data testing solution that is the first-of-its-kind full DevOps solution for continuous data testing. Key features include Robust API with 60+ calls, detailed data intelligence & data analytics, seamless integration into the DevOps pipeline for continuous testing, and verifies large amounts of data quickly.



2) Jenkins

is a Continuous Integration tool which is written using Java language. This tool can be configured via GUI interface or console commands.



Continuous Testing Tools

3) Travis

is continuous testing tool hosted on the GitHub. It offers hosted and on-premises variants. It provides a variety of different languages and a good documentation.

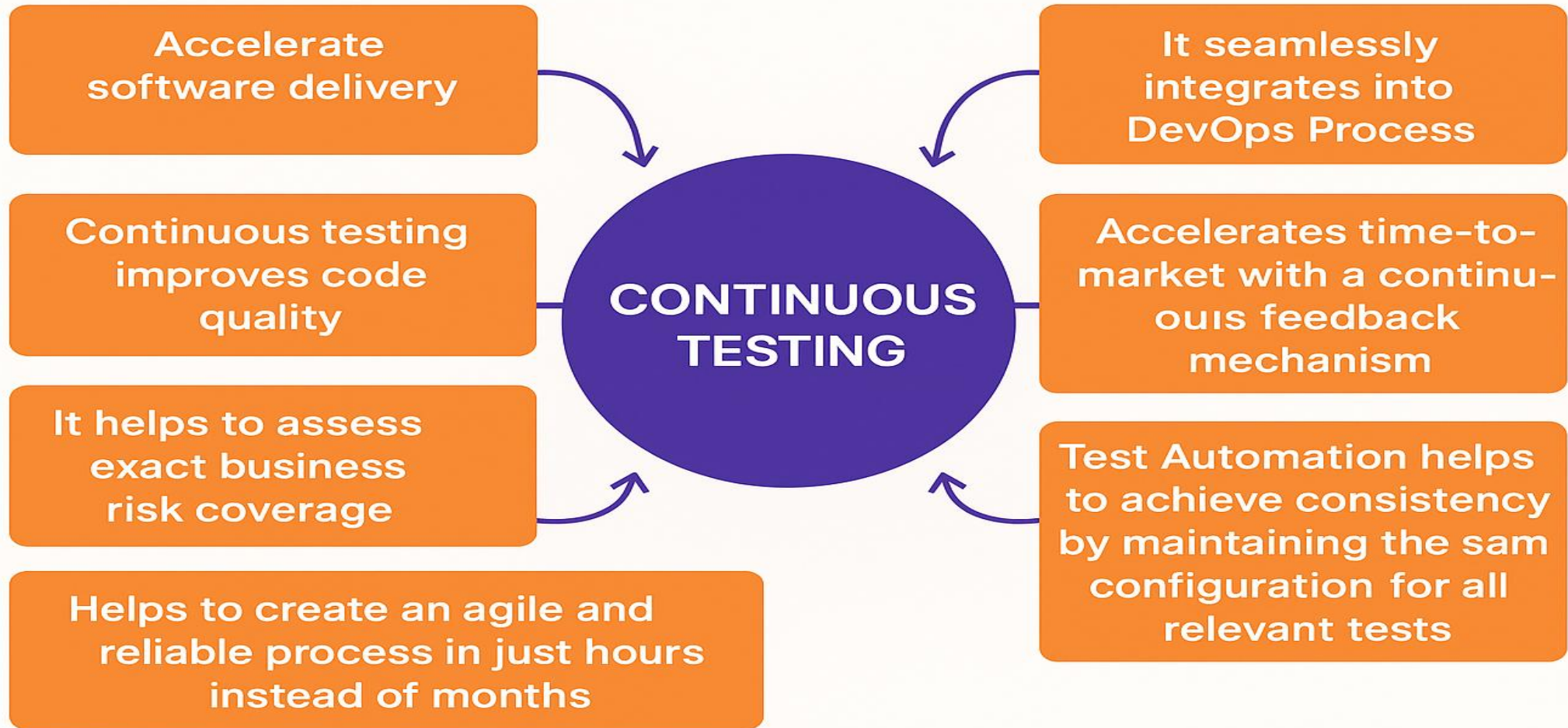


4) Selenium

is open-source software testing tool. It supports all the leading browsers like Firefox, Chrome, IE, and Safari. Selenium WebDriver is used to automate web application testing.



Benefits of Continuous Testing in DevOps



Configuration Management in DevOps

What is configuration Management

- Process by which all environments hosting software are configured and maintained.
- Helps engineering teams build robust and stable systems through the use of tools that automatically manage and monitor updates to configuration data.
- Complex software systems are composed of components that differ in granularity of size and complexity.
- **For eg. microservice architecture.** Each service in a microservice architecture uses configuration metadata to register itself and initialize.

What is configuration Management

keeping all your systems set up the same way — automatically

- You have 10 computers (servers) running a website.
You want all of them to:
- Have the **same software** (e.g., NGINX, Node.js)
- Use the **same settings** (e.g., ports, environment variables)
- Be easy to **update** or **fix** if something breaks

How It's Done?

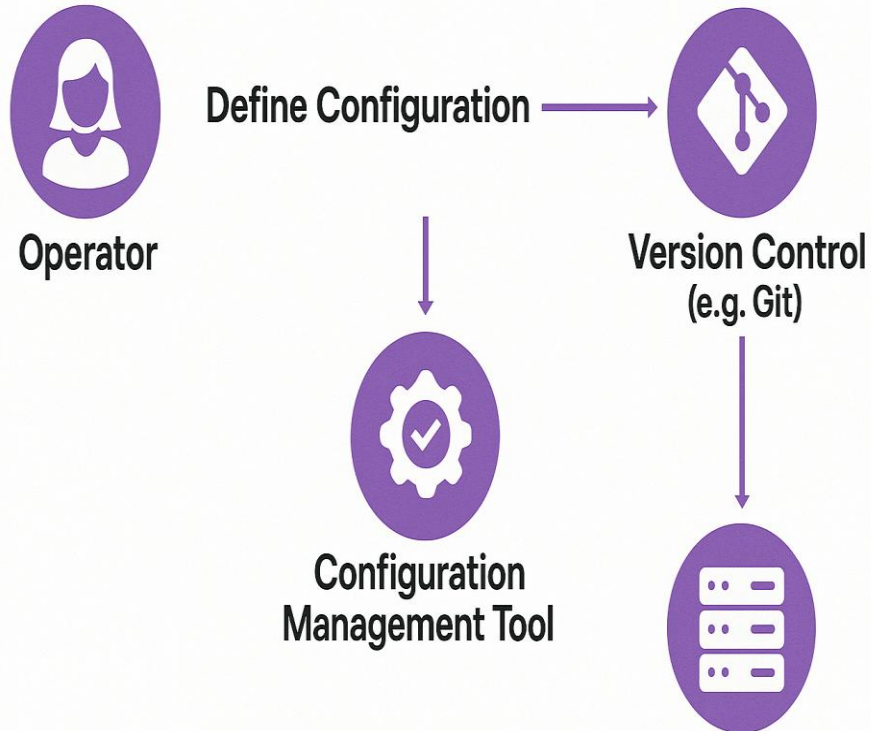
- You write scripts or rules once, and tools like:
 - **Ansible**
 - **Puppet**
 - **Chef**
 - **Terraform** (for cloud)

Apply them to all your servers again and again: exactly the same way.

Configuration Management Helps You:

- **Install the right software** on every server
- **Set the right configuration files** (automatically)
- **Keep everything consistent**, even after updates
- **Speed up deployments** and reduce manual work
- **Avoid errors** that happen from "doing it by hand"

Configuration Management



- **Operator (Admin/DevOps Engineer)**

- This is the person responsible for defining how systems should be configured.

- Writes configuration scripts or declares infrastructure settings (e.g., using YAML, JSON, or domain-specific languages like Puppet, Ansible, or Chef).

- **Define Configuration → Version Control (e.g., Git)**

- The configuration code is stored and version-controlled in Git.

- **Example:** `nginx_config.yaml`, `init_script.sh`, `ansible_playbook.yml`.

- **Configuration Management Tool (e.g., Ansible, Puppet, Chef)**

- These tools fetch the latest configuration from the version control system.

- They **automate** the process of applying those configurations to systems, ensuring consistency.

- **Managed System (Servers, Containers, Cloud Instances, etc.)**

- The configuration tool applies the defined setup to these systems.

- This can include installing software, setting permissions, deploying apps, etc.

What is configuration Management contd...

- Some examples of software configuration metadata are:

- Specifications of computational hardware resource allocations for CPU, RAM, etc.
- Endpoints that specify external connections to other services, databases, or domains

(Used to define how services interact with external systems like databases or APIs.)

- Secrets like passwords and encryption keys

(Stored securely using secret management tools or environment variables, not hard-coded in plain text.)

- A Configuration management (CM) tool assists a firm in enforcing a specific setup value for each item and notifies them of any configuration errors in real-time.

These are typically defined in configuration files like Kubernetes manifests or Docker Compose files.

Example: Kubernetes Pod YAML

resources:

requests:

memory: "512Mi"

cpu: "500m"

limits:

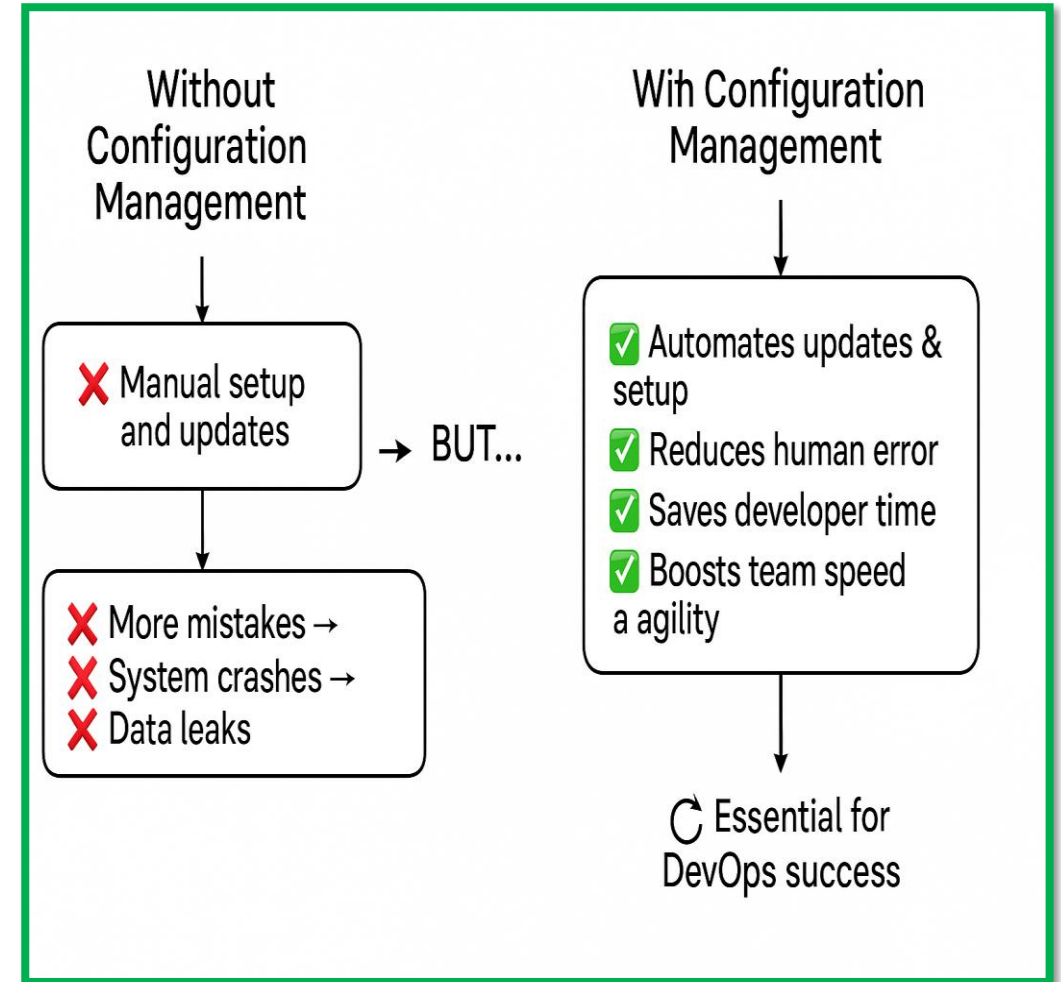
memory: "1Gi"

cpu: "1"

This configuration defines a container that requests 512MiB RAM and 0.5 CPU cores, and limits its usage to 1GiB RAM and 1 full CPU core.

Why need CM in DevOps

- Without proper configuration management, problems can happen:
 - Systems may go **down** (outages)
 - **Sensitive data** can get exposed (breaches)
- With configuration management:
 - Common tasks are **automated**
 - Developers get more time for **actual coding**
 - The whole team becomes more **agile** and **fast**
- It's a **key part** of DevOps to keep systems **reliable** and **secure**



Elements of Configuration Management in DevOps

- **Configuration Identification:** Identify the configuration of the environment to be maintained.
- **Configuration Control:**
 - once the configuration has been identified, it might not remain unchanged.
 - needs to be some kind of mechanism in place to track and control changes to the configuration
- **Configuration Audit:**
 - is a review of the existing systems to ensure that it stands up to compliance regulation and validations.

Primary components required for configuration Management in DevOps

Artifact repository:

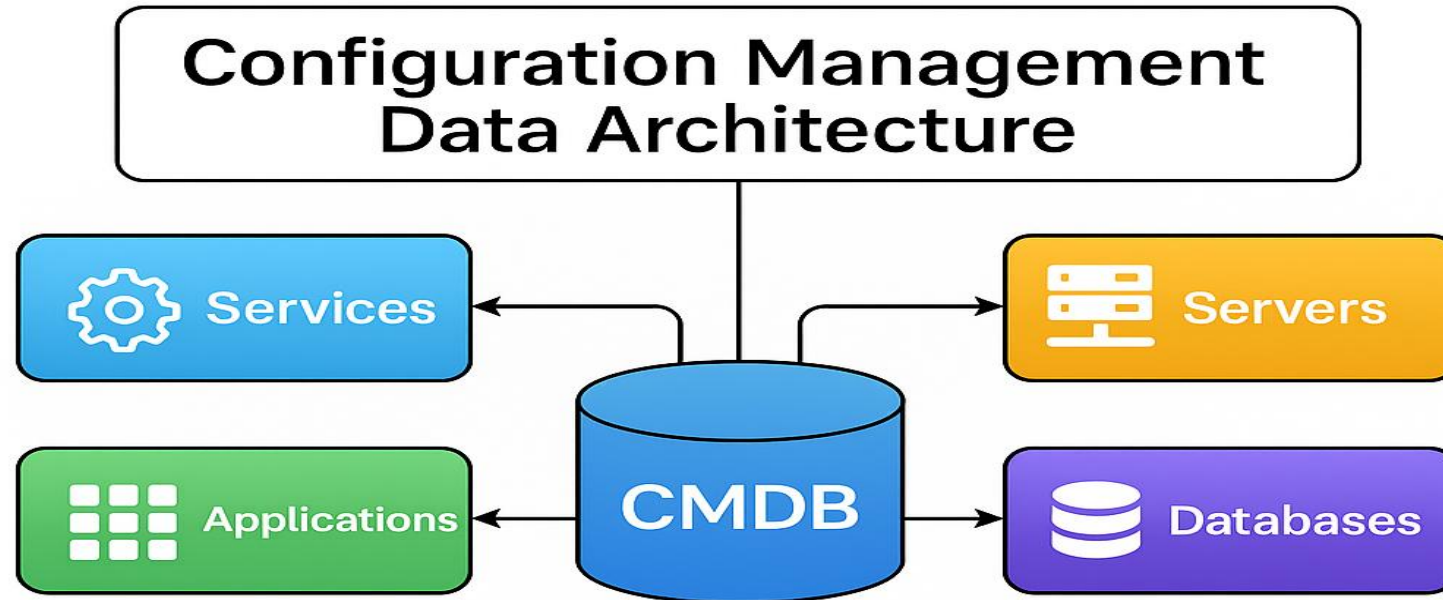
- Meant to store machine files.
- This can include binaries, test data, and libraries.
- In DevOps, artifacts, like binaries, are a natural result of continuous integration.
- DevOps developers are always pushing out builds which, in turn, create artifact files that need to be stored, but not necessarily accessed.

Primary components required for configuration Management in DevOps Contd..

Source code repository

- Database of source code which developers use.
- This database serves as a container for all the working code.
- Source code aside, it stores a number of useful components including various scripts and configuration files.
- two types of source code repositories: **centralized version control system (CVCS)** and **distributed version control system (DVCS)**.
- In a CVCS, the source code lives in a centralized place, where it can be retrieved and stored.
- In DVCS, the code exists across multiple terminals useful in the development process.

Primary components required for configuration Management in DevOps contd..



A configuration management database or (CMDB) is a relational database that spans across multiple systems and applications related to configuration management, including services, servers, applications, and databases to name a few.

CMDB is helpful for change management, as it allows users to audit the relationships between integrated systems before configuration changes are made.

Real-life example of Configuration Management in DevOps using Ansible

Deploying a Web App to Multiple Servers

An e-commerce company needs to deploy its web application on 20 Linux servers (for frontend, backend, and database) across different environments: development, staging, and production.

Without Configuration Management:

- Sys admins manually install and configure:
 - Node.js
 - MongoDB
- They update firewall rules, environment variables, and service settings into each server.
- Result:
 - Inconsistencies,
 - More downtime,
 - Errors like different versions or misconfigured ports.

Real-life example of Configuration Management in DevOps using Ansible contd..

With Configuration Management using Ansible:

- **Ansible Playbooks are created to:**
 - Install and configure NGINX, Node.js, MongoDB
 - Set environment variables
 - Start services
 - Configure load balancer and firewall
- Single command deploys changes to all servers:
bash
CopyEdit
ansible-playbook deploy.yml -i inventory/production
- Version-controlled playbooks allow tracking changes (via Git).
- Automatically applies identical configuration on dev → staging → prod.

Benefit	Result
Faster deployments	From hours to minutes
Consistent setup	Every server matches the exact config
Rollbacks possible	If config fails, revert to last version
Better security	Centralized control of secrets and access
Auditable changes	Logs show who changed what, when

Tools Commonly Used:

- **Ansible** (agentless, YAML-based)
- **Puppet/Chef** (used in large-scale enterprises)
- **Terraform** (for infra + config management in cloud)
- **SaltStack**

Best Configuration Management Tools in DevOps

1. Ansible

Ansible is an open-source application deployment, software provisioning, and configuration management tool. Ansible works with multiple systems of your infrastructure by considering ansible's inventory and ASCII versionable text files. You can pull multiple inventories from cloud sources and formats(INI, YMAL) at the same time.



2. CFEngine

CFEngine is an automation tool used for provisioning and managing software deployment in the operational IT environment. The software of CFEngine is written in C, and it works in a client-server mode with the help of remote nodes and server managing agents. CFEngine is an independent operating system interface. It can host on windows, Unix as well Linux, AIX, HP-UX, and Solaris.



Best Configuration Management Tools in DevOps contd..

3. Puppet

Gives an automated way to inspect, deliver, and operate your software deployment. It enables system administrators to create control and consistency across the infrastructure, with common easy language. Testing, sharing, and enforcing the modifications in the data center and cloud platforms has become very easy with a puppet.



4. RackHD

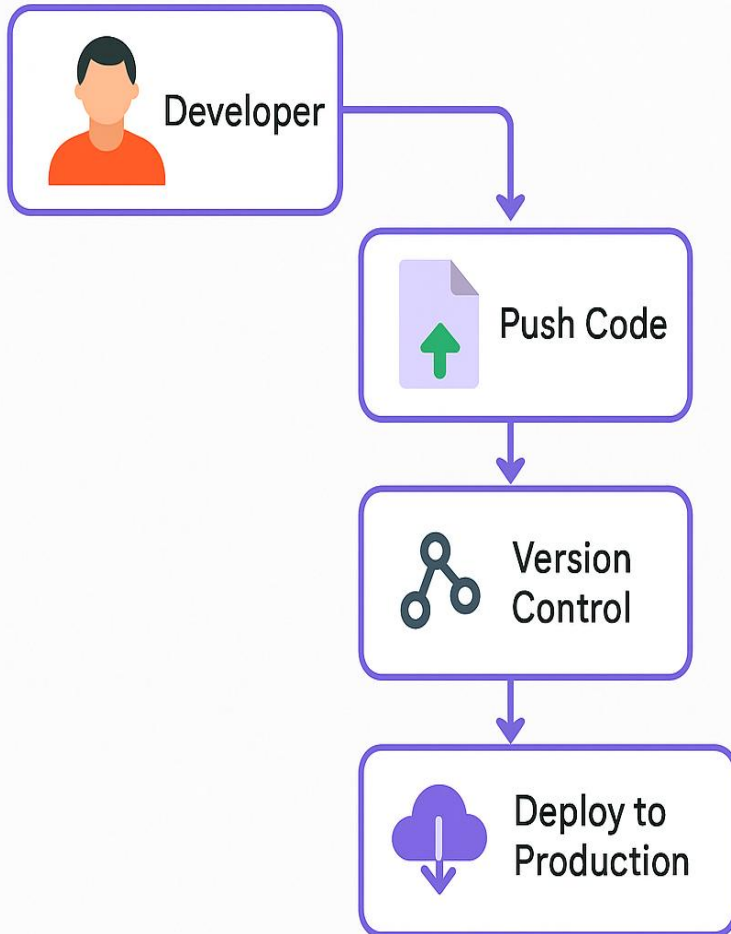
Automated hardware management and orchestration tool that works with cohesive APIs. Developers can use APIs provided in RackHD to create an interface for users as single access for managing all the hardware services.



Continuous Deployment in DevOps

- **Continuous deployment is also a continuous delivery pipeline but without any manual gates in between.**
- **No manual gates or no manual intervention, from the initial code commit till the code is into the production.**

Continuous Deployment



1. Developer

1. Writes new features or fixes bugs in the code.

2. Push Code

1. Developer saves the changes and sends (pushes) them to a shared codebase (e.g., GitHub).

3. Version Control

1. Code is stored and tracked in a version control system like Git.
2. This keeps a record of who changed what, and when.

4. Deploy to Production

1. If the code passes all automated tests, it is **automatically deployed** to the **live app or website**, with no human waiting to press "deploy."

Continuous Deployment in DevOps contd..

Real-Life Example: Facebook

- Facebook engineers make many small code updates every day.
- If an engineer fixes a small bug in the mobile app:
 - They write the fix → push the code → it goes through automated testing → and is automatically released to the app.
- Users might not even notice — the app just quietly improves

Continuous Deployment in DevOps contd..

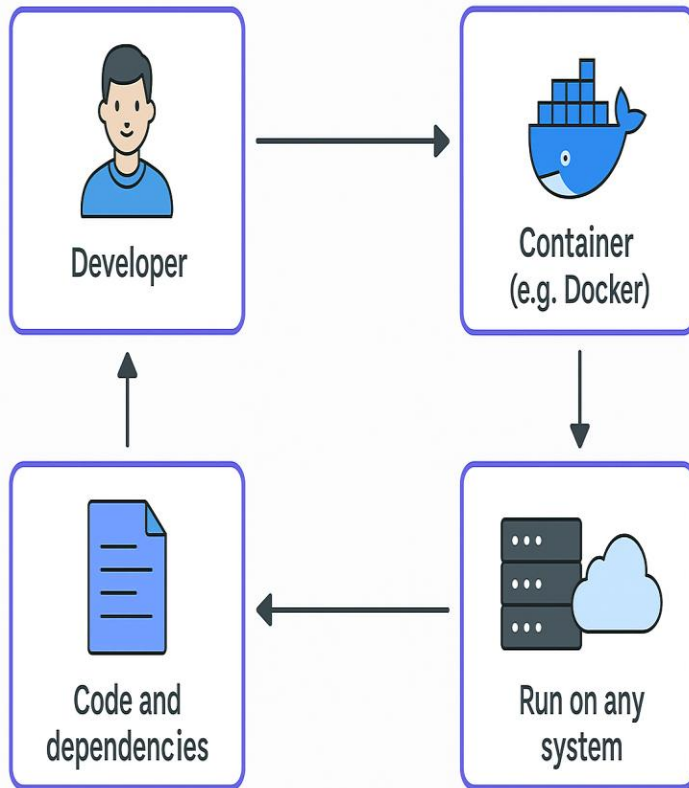
Example:

- **a value-added service offer of a mobile operator to the customers, during Christmas, can be deployed via continuous deployment method to production any time, so that the feature or value-added service package is kept ready on the live and opened up during Christmas or at midnight of the new year etc., through a feature toggle.**
- **And this feature can be opened up in production during wee hours or sleeping hours and safely be turned on to the end users later during the day, after ensuring that it is working fine.**
- **Basically, it's like deploying the code/feature to the production, but not running the code until it is the right time or the team makes a decision to release the code or feature.**
- **This option also allows turning on the feature to a certain set of users or certain geography or certain customer site or even for internal employees for initial evaluation and feedback.**

What Is Containerization?

- **It is a process of packaging your application together with its dependencies into one package (a container).**
- **Such a package can then be run anywhere, no matter if it's an on-premises server, a virtual machine in the cloud, or a developer's laptop.**
- **By abstracting the infrastructure, containerization allows you to make your application truly portable and flexible.**

Containerization in DevOps



1. Developer

1. The developer writes the application code — for example, a web app using Python or Node.js.

2. Container (e.g., Docker)

1. The code is packaged into a **container** using tools like **Docker**.
2. This container includes:
 1. The app code
 2. Runtime (e.g., Node.js engine)
 3. Libraries
 4. Configurations

3. Run on Any System

1. The container can now run **anywhere** whether it's:
 1. A developer's laptop
 2. A testing server
 3. A production cloud platform (e.g., AWS, Azure, GCP)

4. Code and Dependencies

1. Inside the container, all the **code and dependencies** are bundled.
2. This ensures it behaves the **same across all environments**.

Problems of Traditional Applications

- To install and run any software on any server, need to meet several requirements.
- The software needs to support your operating system, and you probably need to have a few libraries and tools installed to run the software.
- All these requirements probably need to be in a specific version and accessible at the specific path.
- Also, sometimes you may need to have proper permissions for certain directories and files.
- Overall, there are quite a few checkboxes you need to tick before you can successfully run the software.

What Does Containerization Mean for DevOps?

Containerization requires placing a software component and its environment, dependencies, and configuration, into an isolated unit called a container. This makes it possible to deploy an application consistently on any computing environment, whether on-premises or cloud-based.

Containerization in DevOps

- **Docker containers are a natural fit for DevOps, providing significant advantages compared to virtualization or bare-metal deployment.**
 - **They are easier and faster to deploy,**
 - **require fewer resources to run,**
 - **are easier to manage,**
 - **are generally more flexible.**
- **These advantages help DevOps teams break applications into microservices,**
- **Each of which can be rapidly updated and deployed, increasing development velocity and improving agility.**
- **Containers allow DevOps teams to standardize the way applications are packaged, delivered and deployed across the development lifecycle.**